The mathcomp-eulerian formalization

A Stanley-style reading of permutation statistics in Rocq/MathComp

Contents

About this document

This document is a Stanley-style reading of the `mathcomp-eulerian` formalization. Each definition and statement is given twice: once in ordinary mathematical English (in the spirit of Stanley *Enumerative Combinatorics I*, Chapter 1) and once as the verbatim Rocq text, with a clickable link to the line in the repository on GitHub. The intention is that a mathematician can read top to bottom like a textbook and at any point click through to verify the formal statement.

Status. This edition covers Stanley §1.3.1–§1.3.4 (cycles, Stirling numbers, inversions, major index, Foata's bijection, MacMahon's equidistribution) and §1.4 (descents, Eulerian numbers, set-refined count $\beta_n(S)$). Chapter 1 is a mathcomp primer; chapters 2–4 cover §1.3; Chapter 5 covers §1.4; Appendix A is a quick-reference glossary. Chapters covering the q-analogues, §1.6.2 (longest alternating subsequence), and §1.6.3 (toggle action and the headline corollary) are planned but not yet written.

First-time readers. Read Chapter 1 once, then Chapter 5 top to bottom. Use Appendix A as a quick reference on second reading. The primer and glossary together contain everything a Stanley reader needs to know about mathcomp to verify the formal statements; you do not need to install Rocq.

Conventions. The single recurring shift is 0-indexing. Stanley writes $w \in \mathfrak{S}_n$ for permutations of $\{1, \ldots, n\}$ and $\mathrm{D}(w) \subseteq \{1, \ldots, n-1\}$ for the descent set; we use $\{\texttt{perm 'I\_n}\}$ for permutations of $\{0, \ldots, n-1\}$ and $\{\texttt{set 'I\_n}_{-1}\}$ for the descent set. Concretely:

$$\text{our } \{\texttt{perm 'I\_n}_{+1}\} \;=\; \text{Stanley's } \mathfrak{S}_{n+1}.$$

A complete translation table is in `docs/READING_GUIDE.md`; a worked example through Stanley's $w = [3, 1, 4, 2]$ is in the same file.

Reading the formal text. Each Rocq block looks like this:

[Rocq, `descent.v:25`]

```
Definition descent_set s : {set 'I_n} := [set i | is_descent s i].
```

Decoded. $\mathrm{D}(s) = \{\, i \in \{0, \ldots, n-1\} : s(i) > s(i+1) \,\}$.

The bracketed link at the top is clickable and opens the exact line on GitHub. The Rocq body is verbatim; identifiers and keywords are highlighted but no mathematical substitution has been performed. The shaded *Decoded* line below the block restates the formal text in plain mathematics, mechanically substituting mathcomp casts for their meaning. Side by side, the two should say the same thing; if they do not, that is exactly the kind of discrepancy worth flagging. Some definitions also carry a Design choice sidebar that explains why a particular formalization was chosen when a more natural-looking alternative exists. These boxes are for first reading and can be skipped on a re-read.

What is *not* included here. Full proofs are not included; only the formal *statements* are shown alongside the informal mathematics, with a one-paragraph proof sketch for each non-trivial result. The full proof is always one click away in the linked source. The cd-index development (`mmtree.v`, `psi_*.v`) is project-internal scaffolding not in Stanley and is not covered.

Verifying axiom-freeness. Every theorem in this document is kernel-validated by Rocq with no axioms or admits. To check this yourself for, e.g., the headline Corollary **??**:

```
echo 'From mathcomp_eulerian Require Import beta_swap.
      Print Assumptions beta_alt_max.' | rocq top -R . mathcomp_eulerian
```

prints `Closed under the global context`, meaning no axiom is used anywhere in its transitive dependencies.

Chapter 1

A combinatorialist's mathcomp primer

This chapter is a focused reference for the few mathcomp primitives that appear in Chapter 5. It is not a tutorial; it gives just enough vocabulary to read the formal blocks and recognise that they say what the informal text claims. Skim it once on first read; come back to it as a glossary when re-reading.

1.1 Ordinals: the type `'I_n`

The type `'I_n` (read "I-sub-n") is mathcomp's *finite type* of natural numbers strictly less than n. An inhabitant is a pair: a natural number `i : nat` and a *proof* `i < n`. The two together are written `Ordinal i (proof : i < n)` but a reader normally never constructs ordinals by hand — the type appears as the index of a sum or a permutation domain, and inhabitants come from the typing context.

`ord0 : 'I_n.+1` the value 0, with its proof $0 < n + 1$. Always available when the bound is at least 1.

`ord_max : 'I_n.+1` the value n, with its proof. The largest element of `'I_n.+1`.

`val i : nat` strips an ordinal back to its underlying natural number, forgetting the proof.

The *cardinality* of `'I_n` is exactly n (lemma `card_ord` in `fintype.v`). So `#|'I_4| = 4`.

1.2 Casts: `widen_ord`, `lift`, `unlift`

There are exactly two natural maps $'\mathtt{I}_n \to '\mathtt{I}_{n+1}$:

`widen_ord (h : n \leq n+1) i` sends `i : 'I_n` to the same numerical value, viewed inside the bigger type. Diagrammatically:

$$i : '\mathtt{I}_n \quad \xrightarrow{\texttt{widen\_ord}} \quad i : '\mathtt{I}_{n+1}.$$

The h argument is a proof that $n \leq n + 1$, supplied by `leqnSn n`. Mathcomp users write `widen_ord (leqnSn n) i` reflexively.

`lift (p : 'I_n.+1) i` sends `i : 'I_n` to a value in `'I_n.+1` *skipping* the value p. With $p =$ `ord0`, this shifts every value up by one:

$$i : 'I_n \quad \xrightarrow{\texttt{lift ord0}} \quad i+1 : 'I_{n+1}.$$

The general `lift p i` skips p, which is exactly what one needs when removing one position from a permutation.

The two casts together name two specific elements of `'I_n.+1` from the same source $i : 'I_n$: the value i itself, and the value $i+1$. This is the idiom that lets us write "the comparison between consecutive positions i and $i+1$" without ever leaving the well-typed world of finite ordinals.

The partial inverse of `lift p` is `unlift p`, of type `'I_n.+1 \to option 'I_n`. Concretely: `unlift p i = None` if `i = p`, and `unlift p i = Some j` otherwise, with j the unique witness of `i = lift p j`. This is the canonical Rocq idiom for the case-split "is the input the special index p, or one of the others?" — it appears in every inductive bijection on ordinals.

1.3 Finite sets: `{set T}`

For `T` a finite type (such as `'I_n`), `{set T}` is the type of *finite subsets of* T, with *extensional* equality: two sets are equal iff they have the same elements.

`[set i : T | P i]` set comprehension — the set of `i : T` satisfying the boolean predicate P. The vertical bar reads as "such that", as in mathematical notation.

`#|S|` the cardinality of `S` as a natural number.

`i \in S` membership test, returning a `bool`.

`~: S` complement, `S :|: T` union, `S :&: T` intersection, `S :\: T` difference.

`{set T}` is itself a finite type, which means big operators (sums, products) can be indexed by sets. This matters for the Eulerian recurrence and for the partition-by-descent-set identity `beta_eulerian`.

1.4 Permutations: `{perm T}`

`{perm T}` is the type of *bijections* `T \to T` for `T` a finite type. An inhabitant is a function together with a proof of bijectivity, packaged so that:

`s i` applies the permutation `s` to `i : T` as a function. (No special syntax is needed: `{perm T}` *coerces* to a function.)

`1%g` is the identity permutation. The `%g` selects mathcomp's group scope.

`(s * t)%g i` is composition. Mathcomp's convention is that `(s * t) i = t (s i)` (right-to-left for the *action*; the lemma is `permM`).

`perm (h : injective f)` constructs a permutation from an injective function `f : T \to T` (on a finite type, injective implies bijective).

The cardinality of `{perm 'I_n}` is $n!$ (Stanley's $|\mathfrak{S}_n|$); this is the lemma `card_perm` or `card_Sn` in mathcomp.

1.5 Big operators: \sum, \prod

Mathcomp's `bigop.v` provides indexed sums and products with a uniform syntax:

`\sum_(i : T) f i` sums `f i` over every `i : T` (`T` a finite type).

`\sum_(i < n) f i` same, with `i : 'I_n`.

`\sum_(i \in S) f i` sums over an explicit set `S`.

`\sum_(i | P i) f i` sums over indices satisfying the boolean filter `P`.

The vertical bar in `\sum_(i | P i)` is a *filter*, not the disjunction symbol. Replace `\sum` by `\prod` for products. Several recurring *moves* appear in the proofs:

`partition_big f xpredT` rewrite `\sum_x g x` as `\sum_y \sum_(x | f x = y) g x` — partition by the value of f.

`bigID p` split a sum into the part where the predicate p holds and the part where it does not.

`reindex ...` change of summation variable along a bijection.

These names appear in the proofs we will not reprint in this document, but they are worth recognising if you click through to a `.v` file.

1.6 Sections and section variables

Every formalization file in this project opens with

```
Section X.
Variable n : nat.
Definition foo := ...     (* implicitly takes n *)
Lemma bar : ...           (* implicitly takes n *)
End X.
```

Inside the section, definitions and lemmas may use `n` as if it were globally fixed. At `End X`, every definition and lemma is automatically generalised to take `n` as an explicit argument. This is why `descent.v` reads `Variable n : nat` once at the top and then `Definition is_descent s i := ...` without an explicit `n`, but outside the file `is_descent` is invoked as `is_descent n s i`. (For the mathematician: think of `Variable` as "Let `n` be a natural number" at the start of a chapter.)

1.7 The takeaway

Three points are enough to read Chapter 5:

1. `{perm 'I_n.+1}` is mathcomp's $\mathfrak{S}_{n+1}$. Apply a permutation to a position by writing `s i`; the descent set is a `{set 'I_n}` (positions, not values).
2. The two casts `widen_ord (leqnSn n) i` and `lift ord0 i` pick out positions i and $i+1$ inside `'I_n.+1` from a single $i : \mathtt{'I_n}$. The reader does not need to verify cast arithmetic by hand — the kernel rejects the file unless every cast is the unique one of its type.
3. Big operators `\sum_(...)` are just sums; the syntax variants control the index range. The vertical bar is a filter.

With these in hand, the formal blocks in Chapter 5 read directly. Each formal block is followed by a one-line *Decoded* reading that mechanically substitutes the mathcomp casts for their mathematical meaning, so the reader has both forms side by side.

Chapter 2

Cycles and Stirling numbers

This chapter formalizes Stanley §1.3.1 (cycle representation of permutations) and §1.3.2 (Stirling numbers of the first kind). The two main objects are the cycle count $c(w)$ and the (signless) Stirling cycle number $c(n,k)$ that counts permutations by $c(w)$. They are the cycle-side analogues of $\mathrm{des}(w)$ and $A(n,k)$ from Chapter 5.

2.1 Cycle count

Definition 2.1 (Cycle decomposition, cycle count)**.** Every $w \in \mathfrak{S}_n$ admits a unique decomposition into a set of *disjoint cycles*; let $c(w)$ denote the number of such cycles.

[Rocq, cycles.v:24]

```
Definition cycle_count {T : finType} (s : {perm T}) : nat :=
  #|porbits s|.
```

Decoded. $c(s)$ = number of orbits of the action of the cyclic group $\langle s \rangle$ on the underlying set — that is, the number of cycles in the disjoint-cycle decomposition.

Design choice: why `porbits`, not a manual cycle decomposition?

A naive formalization would build the cycle decomposition by hand: pick an element, follow the orbit, mark seen, repeat. Mathcomp's `porbits` (in `perm.v`) does this once and exposes the result as a `{set {set T}}` — a set of orbits — together with all the lemmas one would want (`porbits_partition`, `cover_porbits`, etc.). Reusing it costs nothing and gives proofs like `cycle_count_id` in three lines instead of thirty. The cardinality of the orbit set is exactly the cycle count.

Lemma 2.2 (Cycle count bound)**.** $c(w) \le |T|$ *for every* $w \in \mathfrak{S}_T$*, with equality iff* w *is the identity.*

[Rocq, cycles.v:44]

```
Lemma cycle_count_le {T : finType} (s : {perm T}) :
  cycle_count s ≤ #|T|.
```

Decoded. The number of cycles is at most the number of elements (each cycle has at least one element).

[Rocq, cycles.v:64]

```
Lemma cycle_count_id (T : finType) :
  cycle_count (1 : {perm T}) = #|T|.
```

Decoded. The identity has every element as a fixed point, i.e. a 1-cycle; the cycle count equals $|T|$.

2.2 Stirling cycle numbers

Definition 2.3 (Signless Stirling cycle numbers)**.** The number $c(n, k)$ is the count of permutations of $[n] = \{1, \ldots, n\}$ with exactly k cycles.

[Rocq, cycles.v:33]

```
Definition stirling_c (n k : nat) : nat :=
  #|[set s : {perm 'I_n} | cycle_count s == k]|.
```

Decoded. $c(n, k) = |\{s \in \mathfrak{S}_n : c(s) = k\}|$, the number of perms of length n with k cycles. *Direct match to Stanley* (no off-by-one): we use `{perm 'I_n}`, of size n, exactly mirroring Stanley's S_n.

Lemma 2.4 (Row sum recovers $n!$)**.** $\sum_{k=0}^{n} c(n, k) = n!$

[Rocq, cycles.v:98]

```
Lemma stirling_c_row_sum_fact n :
  \sum_(k < n.+1) stirling_c n k = n`!.
```

Decoded. Sum over $k \in \{0, \ldots, n\}$ of $c(n, k)$ equals $n!$, since the sets partition $\mathfrak{S}_n$ by cycle count.

Proof sketch. The sets $\{s : c(s) = k\}$ for k ranging over all possible cycle counts partition $\mathfrak{S}_n$. Total cardinality is $|\mathfrak{S}_n| = n!$. □

Theorem 2.5 (Stirling cycle recurrence)**.** *For $n, k \geq 0$,*

$$c(n+1, k+1) \;=\; n \cdot c(n, k+1) \;+\; c(n, k).$$

[Rocq, stirling_fiber.v:341]

```
Lemma stirling_c_rec n k :
  stirling_c n.+1 k.+1 = n * stirling_c n k.+1 + stirling_c n k.
```

Decoded. Stanley's recurrence for the signless Stirling cycle numbers: a permutation of $[n+1]$ with $k+1$ cycles either inserts the element $n+1$ into one of the existing n cycle-positions of a perm of $[n]$ with $k+1$ cycles (n choices), or sits as a new fixed point in a perm of $[n]$ with k cycles.

Proof sketch. Partition perms of $[n+1]$ with $k+1$ cycles by whether $n+1$ is a fixed point. If yes, the rest is a perm of $[n]$ with k cycles (giving $c(n, k)$). If no, $n+1$ sits inside an existing cycle; there are n positions to insert it, giving $n \cdot c(n, k+1)$.

The formal proof uses an auxiliary construction `insert_after` (in `stirling_fiber.v`) that realises the bijection $[n] \times \{s \in \mathfrak{S}_n : c(s) = k+1\} \to \{\sigma \in \mathfrak{S}_{n+1} : \sigma(n+1) \neq n+1,\ c(\sigma) = k+1\}$. □

Chapter 3

Inversions and the major index

This chapter formalizes Stanley §1.3.3 (the inversion count inv and the major index maj, the two classical Mahonian statistics) and the basic identities each satisfies. The headline result — their equidistribution over $\mathfrak{S}_n$ — requires the Foata bijection and is the subject of Chapter 4.

3.1 Inversions

Definition 3.1 (Inversion)**.** A pair (i, j) with $1 \le i < j \le n$ is an *inversion* of $w \in \mathfrak{S}_n$ if $w_i > w_j$.

[Rocq, inversions.v:26]

```
Definition is_inv s (i j : 'I_n.+1) : bool :=
  (i < j) && (s j < s i).
```

Decoded. (i, j) is an inversion of s iff $i < j$ and $s(j) < s(i)$ (i.e. $s(i) > s(j)$).

Definition 3.2 (Inversion set, inversion count)**.** $\mathrm{Inv}(w) = \{(i, j) : i < j,\ w_i > w_j\}$, and $\mathrm{inv}(w) = |\mathrm{Inv}(w)|$.

[Rocq, inversions.v:30]

```
Definition inv_set s : {set 'I_n.+1 * 'I_n.+1} :=
  [set ij | is_inv s ij.1 ij.2].
```

Decoded. The set of inversion pairs.

[Rocq, inversions.v:34]

```
Definition inv s : nat := #|inv_set s|.
```

Decoded. $\mathrm{inv}(s)$ is the cardinality of the inversion set.

Lemma 3.3 (Identity has no inversions)**.** $\mathrm{inv}(\mathrm{id}) = 0$.

[Rocq, inversions.v:41]

```
Lemma inv_id : inv (1 : {perm 'I_n.+1}) = 0.
```

Decoded. The identity permutation has no (i, j) with $s(i) > s(j)$, hence no inversions.

Lemma 3.4 (Inversion bound)**.** $\mathrm{inv}(w) \le \binom{n}{2}$ *for every* $w \in \mathfrak{S}_n$.

[Rocq, inversions.v:136]

```
Lemma inv_le s : inv s ≤ 'C(n.+1, 2).
```

Decoded. $\mathrm{inv}(s) \le \binom{n+1}{2}$, the number of pairs (i, j) with $i < j$ in $\mathfrak{S}_{n+1}$.

Lemma 3.5 (Reversal identity for inv)**.** $\mathrm{inv}(w^{\mathrm{rev}}) = \binom{n}{2} - \mathrm{inv}(w)$.

[Rocq, inversions.v:219]

```
Lemma inv_rev_perm s :
  inv (rev_perm s) = 'C(n.+1, 2) - inv s.
```

Decoded. The reverse permutation has the "opposite" inversion count: each pair (i, j) with $i < j$ is either an inversion of s or of its reverse, but not both. Hence the two counts sum to $\binom{n+1}{2}$.

3.2 The major index

Definition 3.6 (Major index)**.** The *major index* of $w \in \mathfrak{S}_n$ is the sum of (1-indexed) descent positions:

$$\mathrm{maj}(w) \;=\; \sum_{i \in \mathrm{D}(w)} i.$$

[Rocq, inversions.v:81]

```
Definition maj s : nat := \sum_(i in descent_set s) (val i).+1.
```

Decoded. $\mathrm{maj}(s) = \sum_{i \in \mathrm{D}(s)} (i + 1)$. The `(val i).+1` adds 1 to recover Stanley's 1-indexing from our 0-indexed descent positions.

Design choice: why `(val i).+1` inside the sum, not in the descent set?

We could "correct" the indexing once for all by storing 1-indexed positions in the descent set. We don't, because the descent set type `{set 'I_n}` would no longer fit the ambient type system: 1-indexed positions in $\{1, \ldots, n-1\}$ would need `{set 'I_n.-1}` with values shifted by 1, and every set-level lemma (complement, partition, big-op) would carry a translation. Doing the index shift inside `maj` keeps the descent-set machinery clean and isolates the translation to the one definition that needs it. The price is a single `(val i).+1` in the formula; the benefit is that `descent_set`, `beta`, and `set_is_alt` all live in the same type without conversion casts.

Lemma 3.7 (Major-index bound)**.** $\mathrm{maj}(w) \le \binom{n}{2}$ *for every* $w \in \mathfrak{S}_n$.

[Rocq, inversions.v:158]

```
Lemma maj_le s : maj s ≤ 'C(n.+1, 2).
```

Decoded. $\mathrm{maj}(s) \le \binom{n+1}{2}$, attained by the reverse identity.

Example 3.8 (Stanley's $w = [3, 1, 4, 2]$, statistics)**.** Take $w = [3, 1, 4, 2] \in \mathfrak{S}_4$ as in Example 5.3. Pairs (i, j) with $w_i > w_j$: $(1, 2)$ since $3 > 1$, $(1, 4)$ since $3 > 2$, $(3, 4)$ since $4 > 2$. Hence $\mathrm{inv}(w) = 3$. Descent positions: $\mathrm{D}(w) = \{1, 3\}$, so $\mathrm{maj}(w) = 1 + 3 = 4$. In our types, $\mathrm{inv}(s) = 3$ and $\mathrm{maj}(s) = (0 + 1) + (2 + 1) = 4$, matching.

Chapter 4

Foata's bijection and MacMahon's equidistribution

This chapter formalizes Stanley §1.3.4: Foata's first fundamental bijection on $\mathfrak{S}_n$, which sends permutations of major index k bijectively to permutations of inversion count k. The corollary is MacMahon's equidistribution theorem.

4.1 The Foata transformation on words

The bijection is built from a recursive construction on words. Given a word u and a new letter a, classify the previous letters of u according to whether they are $< a$ or $> a$ (depending on the last letter of u), split u into blocks at the classifying letters, and cyclically rotate each block (last $\to$ front). Then append a.

[Rocq, foata.v:70-77]

```
Definition foata_step (a : nat) (u : seq nat) : seq nat :=
  match u with
  | [::] ⇒ [:: a]
  | _ :: _ ⇒
      let x := last 0 u in
      let P := if x < a then (fun y : nat ⇒ y < a) else (fun y ⇒ a
          < y) in
      flatten (map cyc_last_to_front (split_blocks P u)) ++ [:: a]
  end.
```

Decoded. Empty word: just $[a]$. Otherwise: pick the predicate P ("$< a$" or "$> a$" depending on the sign of last(u) vs a), `split_blocks P u` cuts u into blocks ending at P-letters, `cyc_last_to_front` rotates each block (last letter to front), `flatten` concatenates, then append a.

[Rocq, foata.v:80]

```
Definition foata (w : seq nat) : seq nat :=
  foldl (fun u a ⇒ foata_step a u) [::] w.
```

Decoded. Process w left-to-right, applying `foata_step` at each new letter starting from the empty word.

Example 4.1 (Stanley's running example)**.** $w = [3, 1, 4, 5, 9, 2, 6]$. Then:

$$\text{foata}(w) \;=\; [3, 4, 1, 5, 2, 9, 6].$$

Verified inside `foata.v` (lemma `sanity_inv_eq_maj` at line 125 confirms by `Compute`). Stanley's hand computation on p. 24 gives the same value.

4.2 Lifting to permutations

The seq-level bijection is wrapped into a permutation via `seq_to_perm`, which packages a uniqueness-and-bound proof obligation into a `{perm 'I_n.+1}`.

[Rocq, foata.v:1529]

```
Definition foata_perm (s : {perm 'I_n.+1}) : {perm 'I_n.+1} :=
  seq_to_perm (foata_perm_to_seq_size s) (foata_perm_to_seq_uniq s)
              (foata_perm_to_seq_bnd s).
```

Decoded. `foata_perm` s is the permutation whose one-line sequence is foata applied to the one-line sequence of s. The three named lemmas discharge the side-conditions (size, distinctness, bound) needed to package a `seq nat` back into a `{perm}`.

Design choice: why a seq-level bijection lifted to perms?

Foata's bijection is defined inductively on the word (seq), not on the permutation as a function. Trying to define it directly on `{perm 'I_n.+1}` would force every step to re-establish "the result is a permutation," an obligation that the seq-level construction discharges once at the end. We pay for the wrapping once (in `seq_to_perm`); in exchange the bijection itself has the natural, recursive form Stanley describes.

4.3 The equidistribution theorem

Theorem 4.2 ($\mathrm{inv}(\mathrm{foata}\, w) = \mathrm{maj}(w)$)**.** *For every* $w \in \mathfrak{S}_n$,

$$\mathrm{inv}(\mathrm{foata}(w)) \;=\; \mathrm{maj}(w).$$

[Rocq, foata.v:1576]

```
Lemma foata_perm_inv_maj n (s : {perm 'I_n.+1}) :
  inv (foata_perm s) = maj s.
```

Decoded. The Foata transformation converts maj-statistics to inv-statistics: for every s, the inversion count of foata(s) equals the major index of s.

Lemma 4.3 (`foata_perm` is injective)**.** *The map* $s \mapsto \mathrm{foata\_perm}\, s$ *is injective on* $\mathfrak{S}_{n+1}$.

[Rocq, foata.v:1586]

```
Lemma foata_perm_inj n : injective (@foata_perm n).
```

Decoded. Different inputs give different outputs (proven by constructing an explicit inverse `foata_inv` at the seq level).

Theorem 4.4 (MacMahon's equidistribution)**.** *For every* n, k*, the number of permutations of* $\mathfrak{S}_n$ *with* $\mathrm{inv} = k$ *equals the number with* $\mathrm{maj} = k$*:*

$$\#\{w \in \mathfrak{S}_n : \mathrm{inv}(w) = k\} \;=\; \#\{w \in \mathfrak{S}_n : \mathrm{maj}(w) = k\}.$$

[Rocq, foata.v:1598]

```
Theorem inv_maj_equidistr n k :
  #|[set s : {perm 'I_n.+1} | inv s == k]|
  = #|[set s : {perm 'I_n.+1} | maj s == k]|.
```

Decoded. The two count functions inv and maj have the same distribution over $\mathfrak{S}_{n+1}$.

Proof sketch. The map $s \mapsto \mathrm{foata\_perm}\, s$ sends $\{s : \mathrm{maj}(s) = k\}$ *into* $\{t : \mathrm{inv}(t) = k\}$ (Theorem 4.2); it is injective; on a finite set, injective implies surjective. So the two sets are in bijection, hence have equal cardinality. □

Chapter 5

Descents and Eulerian numbers

This chapter formalizes Stanley §1.4 (Stanley *Enumerative Combinatorics I*, 2nd ed., pp. 30–38). The central objects are the *descent set* of a permutation, the *Eulerian numbers* $A(n,k)$ that count permutations by descent count, and the set-refined count $\beta_n(S)$ that counts permutations by their full descent set.

5.1 The descent set

Definition 5.1 (Descent)**.** Let $w = (w_1, w_2, \ldots, w_n) \in \mathfrak{S}_n$. A position $i \in \{1, \ldots, n-1\}$ is a *descent* of w if $w_i > w_{i+1}$.

[Rocq, `descent.v:22`]

```
Definition is_descent s i : bool :=
  s (widen_ord (leqnSn n) i) > s (lift ord0 i).
```

Decoded. $s(i) > s(i+1)$ for $i \in \{0, \ldots, n-1\}$ and $s : \{0, \ldots, n\} \to \{0, \ldots, n\}$ a bijection.

Design choice: why `widen_ord` and `lift`, not `s i > s i.+1`?

The naive form `s i > s i.+1` does not type-check. The permutation `s : {perm 'I_n.+1}` expects an argument of type `'I_n.+1`, but a descent slot i lives in `'I_n` (there are n adjacencies for $n+1$ positions). The two casts `widen_ord (leqnSn n) i` and `lift ord0 i` are the canonical maps that ship i into the larger type `'I_n.+1` as the values i and $i+1$ respectively (Primer 1.2). There is no way to "just cast" — the two casts encode *which* element of `'I_n.+1` we mean. The verbosity is what makes the formalization correct by typing.

The result type `bool` (rather than `Prop`) makes `is_descent` computable: `Compute is_descent s i` returns `true` or `false` for any concrete `s` and `i`. Mathcomp's small-scale reflection bridges `is_descent s i = true` with the corresponding proposition on demand.

Definition 5.2 (Descent set)**.** The *descent set* of $w \in \mathfrak{S}_n$ is

$$\mathrm{D}(w) \;=\; \{\, i \in \{1, \ldots, n-1\} : w_i > w_{i+1} \,\}.$$

[Rocq, `descent.v:25`]

```
Definition descent_set s : {set 'I_n} := [set i | is_descent s i].
```

Decoded. $\mathrm{D}(s) = \{\, i \in \{0, \ldots, n-1\} : s(i) > s(i+1) \,\}$.

Design choice: why `{set 'I_n}`, not `pred 'I_n` or `seq 'I_n`?

Three options exist for representing a finite collection of positions: a boolean predicate (`pred T`), a list (`seq T`), or a set (`{set T}`). `{set 'I_n}` is the right choice here for two reasons. First, it has *extensional equality*: two sets are equal iff they have the same elements, so `descent_set s = D` is the condition we want, with no axiom of function extensionality required. Second, `{set 'I_n}` is itself a finite type, which means we can sum over it with mathcomp's big operators — the lemma `beta_eulerian` below uses `\sum_(D : {set 'I_n} | #|D| == k)` exactly because `{set}` supports this. `pred 'I_n` fails the first test (extensional equality of predicates needs `funext`); `seq 'I_n` carries spurious order and multiplicity.

$\mathrm{D}(w)$ is a subset of $\{1, \ldots, n-1\}$ in Stanley; in our types it is a subset of $\{0, \ldots, n-1\} = {}'\mathtt{I_n}$. The correspondence is Stanley's $i \leftrightarrow$ our $(i-1)$.

Example 5.3 (Stanley's $w = [3,1,4,2]$, both ways)**.** Take $w = [3,1,4,2] \in \mathfrak{S}_4$. Then $\mathrm{D}(w) = \{1,3\}$ since $w_1 = 3 > 1 = w_2$ and $w_3 = 4 > 2 = w_4$. In our types, the corresponding $s : \{\mathtt{perm}\ {}'\mathtt{I_4}\}$ has $s\,0 = 2$, $s\,1 = 0$, $s\,2 = 3$, $s\,3 = 1$ (values shifted to $\{0,1,2,3\}$). Then:

```
Compute is_descent s 0.     (* = true, since s 0 = 2 > 0 = s 1 *)
Compute is_descent s 1.     (* = false, since s 1 = 0 < 3 = s 2 *)
Compute is_descent s 2.     (* = true, since s 2 = 3 > 1 = s 3 *)
Compute descent_set s.      (* = [set 0; 2] : {set 'I_3} *)
Compute des s.              (* = 2 *)
```

After the index shift our $\{0,2\}$ corresponds to Stanley's $\{1,3\}$, and $\mathrm{des}(s) = 2 = \mathrm{des}(w)$. The computation is what the kernel does on the elaborated term — the formal definition matches the informal one not by editorial promise but by the test running here on the page.

Definition 5.4 (Descent number, ascent number)**.** $\mathrm{des}(w) = |\mathrm{D}(w)|$ and $\mathrm{asc}(w) = (n-1) - \mathrm{des}(w)$.

[Rocq, descent.v:27]

```
Definition des s : nat := #|descent_set s|.
```

Decoded. $\mathrm{des}(s) = |\mathrm{D}(s)|$, the number of descent positions.

[Rocq, descent.v:29]

```
Definition asc s : nat := n - des s.
```

Decoded. $\mathrm{asc}(s) = n - \mathrm{des}(s)$, the number of ascent positions (here n is the number of descent slots, i.e. Stanley's $n-1$).

Design choice: why is the parameter `n` "perm size minus one"?

A permutation of $n+1$ elements has n adjacencies, so the descent set lives in $\{0, \ldots, n-1\}$ regardless of how we parameterise. Two choices: name the parameter after the perm size (forcing every descent-set type to be `{set 'I_n.-1}` with a `.-1`), or name it after the descent slot count (forcing the perm type to be `{perm 'I_n.+1}` with a `.+1`). We picked the latter because it removes a `.-1` from every theorem about descent sets and from every `eulerian n k` statement. The cost is a single conversion: **our parameter `n` is Stanley's** $n-1$, and our `eulerian n k` is Stanley's $A(n+1,k)$. This is a project convention, not a mathcomp requirement; mathcomp itself uses both styles in different files. We flag the off-by-one explicitly at every theorem statement that crosses it.

Lemma 5.5 (Descent count is bounded)**.** $\mathrm{des}(w) \le n-1$ *for every* $w \in \mathfrak{S}_n$.

[Rocq, descent.v:38]

```
Lemma des_le s : des s ≤ n.
```

Decoded. $\mathrm{des}(s) \le n$ for every $s : \{\mathtt{perm}\ {}'\mathtt{I_{n+1}}\}$ (equivalently $\mathrm{des}(w) \le |w| - 1$).

Proof sketch. $\mathrm{D}(w) \subseteq \{0, \ldots, n-1\}$ which has n elements; the cardinality is at most n. □

Lemma 5.6 (Identity has no descents)**.** $\mathrm{des}(\mathrm{id}) = 0$.

[Rocq, descent.v:44]

```
Lemma des_id : des (1 : {perm 'I_n.+1}) = 0.
```

Decoded. $\mathrm{des}(\mathrm{id}) = 0$, where `1%g` is the group identity, i.e. the identity permutation.

Proof sketch. For the identity, $s\,i = i$ at every position, so $s(i) = i < i+1 = s(i+1)$ — no descent slot. □

5.2 Reversal symmetry

A useful symmetry: reversing a permutation flips ascents and descents at corresponding positions.

Definition 5.7 (Reverse permutation)**.** Let $w \in \mathfrak{S}_n$ and define w^{rev} by $w^{\mathrm{rev}}_i = w_{n+1-i}$.

[Rocq, descent.v:80]

```
Definition rev_perm_ord : {perm 'I_n.+1} := perm (@rev_ord_inj
    n.+1).
```

Decoded. The position-reversal involution $i \mapsto n - i$, packaged as a permutation. `perm (h : injective f)` is the constructor that turns an injective function into a `{perm T}`.

[Rocq, descent.v:85]

```
Definition rev_perm s : {perm 'I_n.+1} := rev_perm_ord * s.
```

Decoded. $w^{\mathrm{rev}}(i) = s(n-i)$, the reversal of s viewed at the level of positions.

Design choice: why `rev_perm_ord * s`, not `s * rev_perm_ord`?

Mathcomp's group convention is `(s * t) i = t (s i)` (see the lemma `permM`). With `rev_perm := rev_perm_ord * s`, applying gives `(rev_perm_ord * s) i = s (rev_perm_ord i) = s (n - i)`. This reverses *positions* (Stanley's $w^{\mathrm{rev}}_i = w_{n+1-i}$). Post-composing instead would reverse *values*, a different operation. Picking the wrong side here would silently change the lemma's content; the choice matches Stanley by direct check on a small example.

Lemma 5.8 (Descents of the reverse)**.** *Position i is a descent of w^{rev} iff position $n-i$ is* not *a descent of w:*

$$i \in \mathrm{D}(w^{\mathrm{rev}}) \iff (n-i) \notin \mathrm{D}(w).$$

[Rocq, descent.v:90]

```
Lemma is_descent_rev s (i : 'I_n) :
  is_descent (rev_perm s) i = ~~ is_descent s (rev_ord i).
```

Decoded. Position i is a descent of w^{rev} iff position $n-i$ is *not* a descent of w. The `~~` is boolean negation.

Proof sketch. Direct calculation: $(\texttt{rev\_perm}\,\texttt{s})\,\texttt{j} = s(n-j)$, and the comparison $s(n-i) > s(n-i-1)$ is the negation of $s(n-i-1) > s(n-i)$. □

5.3 Eulerian numbers

Definition 5.9 (Eulerian number)**.** The *Eulerian number* $A(n,k)$ is the number of permutations of $\{1,\dots,n\}$ with exactly k descents:

$$A(n,k) \;=\; \#\{\, w \in \mathfrak{S}_n : \mathrm{des}(w) = k \,\}.$$

[Rocq, eulerian.v:14]

```
Definition eulerian (n k : nat) : nat :=
  #|[set s : {perm 'I_n.+1} | des s == k]|.
```

Decoded. `eulerian` $n\,k = |\{s \in \mathfrak{S}_{n+1} : \mathrm{des}(s) = k\}|$. The `==` is boolean equality on `nat`, and `#|...|` is set cardinality.

Off-by-one alert. The Rocq parameter n is "permutation size minus one" (see the design-choice sidebar above). Concretely, `eulerian n k` counts perms of $\{\texttt{perm 'I}_{n+1}\}$, which are Stanley's $\mathfrak{S}_{n+1}$. Hence

$$\texttt{eulerian}\ n\ k\ =\ A(n+1,k).$$

For instance, `eulerian 3 1` corresponds to Stanley's $A(4,1) = 11$ (Stanley table p. 36).

```
Compute eulerian 0 0.     (* = 1      -- Stanley A(1,0) = 1 *)
Compute eulerian 1 0.     (* = 1      -- Stanley A(2,0) = 1 *)
Compute eulerian 1 1.     (* = 1      -- Stanley A(2,1) = 1 *)
Compute eulerian 2 0.     (* = 1      -- Stanley A(3,0) = 1 *)
Compute eulerian 2 1.     (* = 4      -- Stanley A(3,1) = 4 *)
Compute eulerian 2 2.     (* = 1      -- Stanley A(3,2) = 1 *)
```

The numbers match Stanley's table on p. 36. Computing `eulerian` on `{perm 'I_n.+1}` requires enumerating the set; for $n \geq 4$ the brute-force enumeration is slow, but for the small cases above it returns immediately and the values agree.

Lemma 5.10 (Row sum)**.** $\sum_{k=0}^{n-1} A(n,k) = n!$

[Rocq, eulerian.v:31]

```
Lemma eulerian_row_sum_fact n :
  \sum_(k < n.+1) eulerian n k = n.+1`!.
```

Decoded. $\sum_{k=0}^{n} A(n+1,k) = (n+1)!$. The `` `! `` is mathcomp's factorial notation; `\sum_(k<n.+1)` sums over $k \in \{0,\ldots,n\}$.

Proof sketch. The summands partition $\mathfrak{S}_{n+1}$ by descent count; the total is $|\mathfrak{S}_{n+1}| = (n+1)!$. □

Lemma 5.11 (Out-of-range vanishing)**.** $A(n,k) = 0$ *for* $k \geq n$.

[Rocq, eulerian.v:34]

```
Lemma eulerian_out_of_range n k :
  n < k → eulerian n k = 0.
```

Decoded. If $k > n$ then $A(n+1,k) = 0$, since perms of length $n+1$ have at most n descents.

Proof sketch. A permutation has at most n descent positions (Lemma 5.5); none can have $k > n$ descents. □

Lemma 5.12 (The unique perm with no descents is the identity)**.** *If* $\mathrm{des}(w) = 0$ *then* $w = \mathrm{id}$.

[Rocq, eulerian.v:41]

```
Lemma des0_id n (s : {perm 'I_n.+1}) : des s = 0 → s = 1%g.
```

Decoded. If $\mathrm{des}(s) = 0$ then s is the identity permutation.

Proof sketch. $\mathrm{des}(s) = 0$ means $s(i) < s(i+1)$ for every adjacent pair i, so s is strictly increasing. The only strictly increasing function $\texttt{'I}_{n+1} \to \texttt{'I}_{n+1}$ is the identity. □

5.4 The insert-max bijection

A central tool for the Eulerian recurrence is the bijection that inserts the maximum value $n+1$ at a chosen position into a permutation of $\{1,\dots,n\}$.

Definition 5.13 (Insert-max)**.** For $t \in \mathfrak{S}_n$ and $p \in \{1,\dots,n+1\}$, let $\texttt{insert\_max}(t,p) \in \mathfrak{S}_{n+1}$ be the permutation obtained by inserting the value $n+1$ at position p in the one-line form of t.

[Rocq, eulerian.v:134-157]

```
Definition insert_max_fun (i : 'I_n.+2) : 'I_n.+2 :=
  match unlift p i with
  | Some j ⇒ widen_ord (leqnSn _) (t j)
  | None ⇒ ord_max
  end.

Definition insert_max_perm : {perm 'I_n.+2} := perm
    insert_max_fun_inj.
```

Decoded. For $i \in \{0,\dots,n+1\}$: if $i = p$, output $n+1$ (the new max); otherwise, write $i = \texttt{lift p j}$ for the unique $j \in \{0,\dots,n\}$ that skips p, and output $t(j)$ cast into the larger type.

Design choice: why `match unlift p i with`?

The fundamental case-split for inserting at position p is "is this position p or not?". Mathcomp encodes this as the `option`-valued partial inverse `unlift p`: it returns `None` exactly when $i = p$, and `Some j` otherwise — where $j : \texttt{'I}_{n+1}$ is the unique witness of $i = \texttt{lift p j}$. Carrying j in the `Some` branch (rather than recomputing it from i and p) avoids any arithmetic juggling: we get the correct `'I_n.+1` value with its bound proof for free, and can apply t directly. This is the universal idiom for constructing permutations on `'I_n.+2` from a permutation on `'I_n.+1` plus a special index.

Definition 5.14 (Extract-max)**.** The inverse: given $\sigma \in \mathfrak{S}_{n+1}$ with $\sigma(p) = n+1$ for some position p, remove that value and standardise to obtain $t \in \mathfrak{S}_n$.

[Rocq, eulerian.v:185-203]

```
Definition extract_max_fun (j : 'I_n.+1) : 'I_n.+1 :=
  odflt j (unlift ord_max (s (lift p j))).

Definition extract_max_perm : {perm 'I_n.+1} := perm
    extract_max_fun_inj.
```

Decoded. For $j \in \{0,\dots,n\}$: look at s at position j of the original (skipping p); the result is some value in $\{0,\dots,n+1\}$ that is not $n+1$ (because $\sigma(p) = n+1$ and σ is a bijection); strip the high bit to land in $\{0,\dots,n\}$. The `odflt j (...)` provides a default that is provably unreachable.

Theorem 5.15 (Insert-max is a bijection)**.** *The map* $(t,p) \mapsto \texttt{insert\_max}(t,p)$ *is a bijection* $\mathfrak{S}_n \times \{1,\dots,n+1\} \to \mathfrak{S}_{n+1}$.

[Rocq, eulerian.v:437]

```
Lemma insert_max_perm_bij n :
  ∀ sigma : {perm 'I_n.+2},
    ∃! tp : {perm 'I_n.+1} * 'I_n.+2,
      sigma = insert_max_perm tp.1 tp.2.
```

Decoded. Every $\sigma \in \mathfrak{S}_{n+2}$ arises uniquely as $\texttt{insert\_max}(t,p)$ for some $(t,p) \in \mathfrak{S}_{n+1} \times \{0,\dots,n+1\}$.

Proof sketch. Given σ, the unique p is $\sigma^{-1}(n+1)$ and $t = \texttt{extract\_max}(\sigma,p)$. The two roundtrips are direct calculations. □

5.5 Eulerian recurrence

Theorem 5.16 (Eulerian recurrence)**.** *For $n \geq 1$ and $0 \leq k \leq n$,*

$$A(n+1,k) \;=\; (k+1)\,A(n,k) \;+\; (n-k+1)\,A(n,k-1).$$

[Rocq, eulerian.v:458]

```
Theorem eulerian_rec n k :
  eulerian n.+1 k =
    (k.+1) * eulerian n k + (n.+1 - k) * eulerian n k.-1.
```

Decoded. $A(n+2,k) = (k+1)\,A(n+1,k) + (n+1-k)\,A(n+1,k-1)$. The `n.+1 - k` on the second term reads as "$n+1-k$" under our parameterisation; compare to Stanley's $(n+1-k)$ in $A(n+2,k) = (k+1)A(n+1,k) + (n+1-k)\,A(n+1,k-1)$.

Proof sketch. Using the insert-max bijection, classify each $\sigma \in \mathfrak{S}_{n+1}$ by the position p of $n+1$. There are $k+1$ ascent positions and $n-k$ descent positions where inserting $n+1$ preserves the descent count; inserting at the other $n-k+1$ slots increases it by one. Summing recovers the recurrence. □

5.6 The set-refined count β

The descent count is a coarse invariant. The full descent *set* carries more information; the count of permutations with a fixed descent set is a finer object.

Definition 5.17 (β count)**.** For a subset $S \subseteq \{1,\dots,n-1\}$, the count $\beta_n(S)$ is the number of permutations of $\{1,\dots,n\}$ whose descent set is exactly S:

$$\beta_n(S) \;=\; \#\{\, w \in \mathfrak{S}_n : \mathrm{D}(w) = S \,\}.$$

[Rocq, beta.v:19]

```
Definition beta (n : nat) (D : {set 'I_n}) : nat :=
  #|[set sigma : {perm 'I_n.+1} | descent_set sigma == D]|.
```

Decoded. `beta` $n\,D = |\{\sigma \in \mathfrak{S}_{n+1} : \mathrm{D}(\sigma) = D\}|$, the number of perms of length $n+1$ with descent set exactly D.

Off-by-one alert. As with `eulerian`, the Rocq parameter n is "perm size minus one." So `beta n D` for $D : \{\texttt{set 'I\_n}\}$ equals Stanley's $\beta_{n+1}(D)$, with D interpreted as a subset of $\{0,\dots,n-1\}$ that corresponds to Stanley's 1-indexed $S \subseteq \{1,\dots,n\}$.

Lemma 5.18 (β partitions Eulerian)**.** *Summing $\beta_n(S)$ over all S of cardinality k recovers the Eulerian number:*

$$\sum_{|S|=k} \beta_n(S) \;=\; A(n,k).$$

[Rocq, beta.v:124]

```
Lemma beta_eulerian n k :
  \sum_(D : {set 'I_n} | #|D| == k) beta D = eulerian n k.
```

Decoded. $\sum_{D \subseteq \{0,\dots,n-1\},\ |D|=k} \beta_{n+1}(D) = A(n+1,k)$. The big-op syntax `\sum_(D : {set 'I_n} | #|D| == k)` sums over all sets D of cardinality exactly k.

Proof sketch. The sets $\{\sigma : \mathrm{D}(\sigma) = D\}$ for D ranging over subsets of cardinality k partition the set of permutations with k descents. The cardinalities sum. □

Lemma 5.19 (β is reversal-invariant)**.** *For $S \subseteq \{1, \ldots, n-1\}$, let $S^{\mathrm{rev}} = \{n - i : i \in S\}$. Then $\beta_n(S^{\mathrm{rev}}) = \beta_n(S^{\mathrm{c}})$, where S^{c} is the complement in $\{1, \ldots, n-1\}$.*

[Rocq, beta.v:140]

```
Lemma beta_rev n (D : {set 'I_n}) :
  beta [set rev_ord i | i in D] = beta (~: D).
```

Decoded. $\beta_{n+1}(\{n-1-i : i \in D\}) = \beta_{n+1}(\overline{D})$. The set comprehension `[set rev_ord i | i in D]` is the image of D under $i \mapsto n-1-i$; `~: D` is the complement of D in `'I_n`.

Proof sketch. Reversal $w \mapsto w^{\mathrm{rev}}$ is a bijection on $\mathfrak{S}_n$; by Lemma 5.8 it maps descent set D to $\{n-i : i \notin D\}$, which is the complement of $\{n-i : i \in D\}$. Counting on both sides matches. □

5.7 What this chapter doesn't cover

This chapter establishes the basic objects (D, des, A, β) and the foundational identities. The headline of the formalization,

$$\beta_n(S) \leq E_n \quad \text{with equality iff } S \text{ is alternating} \quad \text{(Stanley Cor 1.6.5)}$$

requires the toggle-action machinery of §1.6.3 and is the subject of a future chapter.

Appendix A

Glossary of mathcomp primitives

A quick-reference companion to Chapter 1. Entries are ordered roughly by first appearance in the chapter; each gives the mathcomp source file, a one-line gloss, and an example.

`'I_n` (`fintype.v`) The finite type of natural numbers strictly less than n. Inhabitant: a pair `(i, proof of i<n)`. Cardinality n.

`ord0 : 'I_n.+1` (`fintype.v`) The value 0, with its proof of $0 < n + 1$.

`ord_max : 'I_n.+1` (`fintype.v`) The value n, the largest element.

`val i : nat` (`eqtype.v`) Strips an ordinal back to its underlying natural number.

`widen_ord (h : m \leq n) i` (`fintype.v`) Casts `i : 'I_m` to `'I_n` keeping the same numerical value. Pure cast.

`lift (p : 'I_n.+1) j` (`fintype.v`) Casts `j : 'I_n` to `'I_n.+1` "skipping" the value p. With $p =$ `ord0`, this is $j \mapsto j + 1$.

`unlift p i : option 'I_n` (`fintype.v`) Partial inverse of `lift p`. Returns `None` when $i = p$, else `Some j` with $i =$ `lift p j`.

`rev_ord i` (`fintype.v`) The involution $i \mapsto n - 1 - i$ on `'I_n`.

`{set T}` (`finset.v`) The type of finite subsets of `T` (a finite type), with extensional equality.

`[set i : T | P i]` (`finset.v`) Set comprehension: `{i : T | P i}`.

`[set f i | i in S]` (`finset.v`) Image of set S under function f.

`#|S|` (`fintype.v`) Cardinality of a finite set or predicate.

`i \in S` (`finset.v`) Membership test, returning `bool`.

`~: S` (`finset.v`) Complement of S in its ambient finite type.

`{perm T}` (`perm.v`) Type of bijections `T \to T` for `T` a finite type. Inhabitant: a function plus its proof of injectivity.

`1%g` (`fingroup.v`) Group identity. For `{perm T}`, the identity permutation.

`(s * t)%g` (`fingroup.v`) Group multiplication. For perms: `(s * t) i = t (s i)`; see lemma `permM`.

`perm (h : injective f)` (`perm.v`) Constructor: turn an injective function into a `{perm T}`.

`\sum_(i : T) f i` (`bigop.v`) Sum of `f i` over every `i : T`.

`\sum_(i < n) f i` (`bigop.v`) Same, indexed by `i : 'I_n`.

`\sum_(i \in S) f i` (`bigop.v`) Sum over the elements of an explicit set `S`.

`\sum_(i | P i) f i` (`bigop.v`) Sum over i satisfying boolean filter P. The vertical bar is a filter, not disjunction.

`partition_big f xpredT` (`bigop.v`) Rewrite a sum as a nested sum partitioned by the value of f.

`n.+1`, `n.+2`, `n.-1` (`ssrnat.v`) Successor, double-successor, predecessor on `nat`. Read as $n+1$, $n+2$, $n-1$.

`~~ b` Boolean negation.

`a == b` (`eqtype.v`) Decidable equality on an `eqType`, returning `bool`. Distinct from propositional `=`; bridged by lemma `eqP`.

`Section X. Variable n. ... End X.` (Rocq) Sections introduce parameters that are auto-generalised on `End`. Inside the section, `n` reads as a fixed natural; outside, every definition is implicitly parameterised.

Appendix B

Source map

A complete file index is in `docs/READING_GUIDE.md` in the `mathcomp-eulerian` repository. Files covered (or planned) by this document, in Stanley reading order:

`cycles.v`, `stirling_fiber.v` Stanley §1.3.1–2 (Chapter 2).

`inversions.v` Stanley §1.3.3 (Chapter 3).

`foata.v` Stanley §1.3.4 (Chapter 4).

`descent.v`, `eulerian.v`, `beta.v` Stanley §1.4 (Chapter 5).

`qfact.v`, `qeul.v` Stanley §1.4 q-analogues (planned).

`altsub.v` Stanley §1.6.2 (planned).

`beta_omega.v`, `beta_swap.v` Stanley §1.6.3 (planned).

`experimental/reflection.v` Stanley §1.6.4 (planned, contains 1 named admit).